MONITORING · DOMAIN 5 - MONITORING AND ALERTING

System tables

Section 5. The platform observing itself, and the traps in reading it.

SECTION 5 of 10 WEIGHT 10% TOPIC System tables

admin@sthithapragna.com

Everything the platform does is written to a table you can query. **Monitoring becomes SQL.**

Five things that are true of all of them

- They live in the **system catalog**, present in every Unity Catalog metastore.
- They are **free**. You pay only for the compute that queries them.
- They are **read-only** and cannot be modified.
- They cover **every workspace in the account in the same region**, not just the one you are in.
- They include data from **non-Unity Catalog workspaces**, even though you need a UC workspace to read them.

> ON THE EXAM
>
> That last pair is the useful one. One query answers a question across every workspace in the region, including ones never migrated to Unity Catalog.

Schemas worth knowing by name

SCHEMA	WHAT IT HOLDS
system.billing	Billable usage, and a historical log of SKU prices.
system.access	Audit events, plus table and column lineage.
system.compute	Cluster and warehouse configuration history, and node utilisation.
system.lakeflow	Jobs, tasks, pipelines and their run timelines.
system.query	Query history for warehouses and serverless compute.
system.storage	Predictive optimization history and related operations.

ON THE EXAM

Cost questions go to billing. Who-did-what goes to access. Why-was-it-slow goes to query and compute. Job outcomes go to lakeflow.

Who can read them

- Account admins and metastore admins have access **by default**.
- Everyone else needs three grants: **USE CATALOG** on system, **USE SCHEMA**, and **SELECT**.
- So access is granted with ordinary Unity Catalog statements, schema by schema.
- That granularity matters. Audit data and billing data rarely suit the same audience.

> ON THE EXAM
>
> Grant per schema rather than per catalog. A finance team needs billing; they almost certainly should not have audit.

Mostly a year, with real exceptions

RETENTION	EXAMPLES
365 days	Most tables, including billing usage, audit, jobs and query history.
180 days	MLflow tables, predictive optimization history.
90 days	Node utilisation timeline, model serving token usage.
30 days	Ingress access denied events.
Indefinite	Price history, node types, workspace metadata.

ON THE EXAM

Node utilisation at 90 days is the one that catches people. A year-on-year capacity comparison is not available from that table.

How a correction appears in billable usage

- When a usage record is wrong, Databricks does not edit it. It adds **two rows**.
- A **retraction** with a **negative** usage quantity that cancels the original, then a **restatement** with the correct values.
- The record type column marks each as original, retraction or restatement.
- So **filtering to original records only** gives you the uncorrected, wrong total.

> ON THE EXAM
>
> A plain sum is correct, because the retraction cancels the original. Being clever and filtering by record type is what breaks it.

When one SKU covers several products

- Some products **bill under the same SKU**. Data quality monitoring, predictive optimization and serverless workflows share one.
- Two columns separate them: **billing origin product** and **product features**.
- So grouping by SKU alone will attribute several different things to one line.
- For job-level cost, join usage metadata to the jobs tables rather than reading the SKU.

> ON THE EXAM
>
> Cost attribution questions usually turn on joining billing to lakeflow or compute. The billing table alone rarely answers 'which job'.

Slowly changing dimensions, and deleted rows

- The jobs, tasks and pipelines tables are **slowly changing dimension type 2**.
- A configuration change emits a **new row**, so the table holds the full history of every entity.
- **A deletion is a new row with a delete time populated**, not the removal of a row.
- So a naive query returns several rows per job. Filter to the current version, or you will double count.

> ON THE EXAM
>
> This is module 8's SCD Type 2 appearing inside the platform's own tables. The same reading rules apply.

Reading system tables incrementally

- Many system tables support streaming reads, which is how you build a live monitoring pipeline.
- **Set skip change commits to true**, or deletes in the system table will disrupt your stream.
- System tables use the default **seven day** vacuum retention.
- So a stream that falls more than seven days behind **breaks**. It is not merely slow.

> ON THE EXAM
>
> A weekly-scheduled stream over a system table is living exactly on the edge of that seven-day window.

Schema drift, and queries that are too broad

- **New columns may be added to system tables at any time.** Existing columns will not change or be removed.
- So a job writing system table data elsewhere should have **schema evolution enabled**.
- A query that is **not selective enough** fails outright, asking for more selective predicates.
- There is also **no real-time monitoring**. Data arrives through the day, so a very recent event may be missing.

> ON THE EXAM
>
> Select star from a system table with no filter is not a slow query. It is an error message.

Do not move this data off the platform

- Databricks **highly discourages** moving system table data outside the platform.
- It can expose sensitive data and put the deployment at risk.
- Audit tables record who accessed what. Lineage records what depends on what.
- If you must export, you are responsible for its security and for preventing misuse.

> ON THE EXAM
>
> An answer that exports audit logs to an external tool for convenience is proposing something the documentation explicitly warns against.

What to carry into the exam

- System catalog, free, read-only, and covering **every workspace in the region**.
- Billing for cost, access for audit and lineage, lakeflow for jobs, query and compute for performance.
- Corrections are a **retraction plus a restatement**. Sum everything; do not filter to original.
- Jobs and pipelines are **SCD Type 2**. Deleted means a delete time, not a missing row.
- Retention is 365 days for most, but **90 for node utilisation** and 30 for ingress denials.

> ON THE EXAM
>
> When a question asks how you would find something out across the whole account, system tables are almost always the answer.

A finance analyst builds a monthly cost report from the billable usage system table. To avoid double counting, they filter to records where the record type is ORIGINAL.

The reported total is consistently higher than the invoice, and the gap varies month to month.

What is wrong?

A Filtering to ORIGINAL excludes the retractions that cancel incorrect records, so uncorrected usage is counted.

B The usage table reports list prices rather than negotiated prices, so it always exceeds the invoice.

C The analyst is missing a filter on the account identifier, so other accounts are included.

D Usage records outside the 365-day retention window are being double counted.

◆ PAUSE AND DECIDE

What is wrong?

A Filtering to ORIGINAL excludes the retractions that cancel incorrect records, so uncorrected usage is counted.

A correction is a negative retraction plus a restatement. Excluding retractions leaves the wrong figures in.

B The usage table reports list prices rather than negotiated prices, so it always exceeds the invoice.

The usage table records quantities. A consistent multiplier would not vary month to month.

C The analyst is missing a filter on the account identifier, so other accounts are included.

System tables cover your own account only.

D Usage records outside the 365-day retention window are being double counted.

Expired records are removed, not duplicated.

ON THE EXAM

The filter was added to prevent double counting and caused the opposite. Sum everything and let the retractions cancel.

A platform team wants a report of cost per job. They join the billable usage table to the jobs table on job identifier and sum the usage quantity.

The totals are far higher than expected, and jobs whose definitions have been edited frequently look the most expensive.

What is causing this?

A Frequently edited jobs genuinely cost more because each edit triggers a full recompute.

B The jobs table is SCD Type 2, so a job with many versions joins to usage many times.

C The usage table stores cost in a different currency for edited jobs.

D Job identifiers are reused after deletion, merging unrelated jobs.

◆ PAUSE AND DECIDE

What is causing this?

A Frequently edited jobs genuinely cost more because each edit triggers a full recompute.

Editing a job definition does not run it.

B The jobs table is SCD Type 2, so a job with many versions joins to usage many times.

Every configuration change emits a new row. Joining without filtering to the current version multiplies the usage.

C The usage table stores cost in a different currency for edited jobs.

Invented.

D Job identifiers are reused after deletion, merging unrelated jobs.

Identifiers are not reused, and deletion is recorded rather than removing the row.

ON THE EXAM

The correlation with edit frequency is the tell. More edits means more rows, and more rows means a larger multiplier.

> A capacity planning exercise needs to compare cluster node utilisation this quarter against the same quarter last year, using the node timeline system table.
>
> The query returns no rows for the earlier period, though the clusters certainly existed then.

What is the explanation?

A The node timeline table has a 90-day retention period, so last year's data no longer exists.

B Node utilisation is only recorded for clusters that are currently running.

C The system schema must be enabled retroactively to backfill historical data.

D Utilisation data is regional, and the clusters ran in another region.

◆ PAUSE AND DECIDE

What is the explanation?

A The node timeline table has a 90-day retention period, so last year's data no longer exists.

One of the documented exceptions to the usual 365 days. A year-on-year comparison is simply not available from it.

B Node utilisation is only recorded for clusters that are currently running.

It records history for terminated clusters too, within retention.

C The system schema must be enabled retroactively to backfill historical data.

There is no retroactive backfill.

D Utilisation data is regional, and the clusters ran in another region.

Possible in general, and it would not produce a clean split at the retention boundary.

ON THE EXAM

Retention decides which questions are answerable at all. Ninety days for node utilisation rules out any year-on-year analysis.

> A security team asks for a monthly extract of the audit system table to be delivered into their own external SIEM tool, so that Databricks activity sits alongside other systems.
>
> A data engineer is asked whether this is a good idea.

What should they say?

A It is straightforward and recommended, since system tables are designed to be exported.

B Databricks highly discourages moving system table data off the platform, and responsibility for its security transfers to whoever exports it.

C It is impossible; system tables cannot be read by anything other than Databricks SQL.

D It is fine provided the extract is limited to the last 30 days.

◆ PAUSE AND DECIDE

What should they say?

A. It is straightforward and recommended, since system tables are designed to be exported.
The documentation says the opposite, in its opening warning.

B. Databricks highly discourages moving system table data off the platform, and responsibility for its security transfers to whoever exports it.
Stated directly. Audit and lineage data is sensitive, and exporting it moves the risk without removing it.

C. It is impossible; system tables cannot be read by anything other than Databricks SQL.
They are ordinary readable tables. The constraint is advisory, not technical.

D. It is fine provided the extract is limited to the last 30 days.
Volume is not the concern. Sensitivity is.

ON THE EXAM

The request is reasonable and the warning is explicit. The useful answer names the trade-off rather than simply refusing.

> A team builds a streaming job that reads the billable usage system table into their own cost dashboard table. It runs on a weekly schedule.
>
> After a public holiday shifts one run by several days, the stream fails and will not restart.

What is the most likely cause?

A. System tables use the default seven-day VACUUM retention, so a stream lagging beyond that breaks.

B. Streaming reads of system tables are only supported for the audit schema.

C. The weekly schedule exceeded the system table's 365-day retention window.

D. The stream must be recreated whenever new columns are added to the source.

◆ PAUSE AND DECIDE

What is the most likely cause?

A System tables use the default seven-day VACUUM retention, so a stream lagging beyond that breaks.

A weekly schedule already sits on the seven-day edge. A few days' delay pushes it past, and the files are gone.

B Streaming reads of system tables are only supported for the audit schema.

Many system tables support streaming, billing usage included.

C The weekly schedule exceeded the system table's 365-day retention window.

Off by two orders of magnitude.

D The stream must be recreated whenever new columns are added to the source.

New columns are a real hazard, and they call for schema evolution rather than causing a hard failure like this.

ON THE EXAM

The design was fragile before the holiday. A weekly stream over a seven-day vacuum window has no margin at all.

> A cost review shows a large and growing line under the serverless jobs SKU. The team cannot account for it from their job schedule, which has not changed.
>
> They are asked to identify what is actually driving the spend.

What should they do?

A. Group by SKU alone, since the SKU uniquely identifies the product.

B. Break the usage down by billing origin product and product features, since several products share that SKU.

C. Check the price history table, since a SKU price increase would explain it.

D. Compare against the node utilisation table to find over-provisioned clusters.

◆ PAUSE AND DECIDE

What should they do?

A Group by SKU alone, since the SKU uniquely identifies the product.

It does not. Several products share the serverless jobs SKU.

B Break the usage down by billing origin product and product features, since several products share that SKU.

Data quality monitoring, predictive optimization and serverless workflows all bill under it. These two columns separate them.

C Check the price history table, since a SKU price increase would explain it.

Worth a glance, and it would not explain growth when the schedule is unchanged.

D Compare against the node utilisation table to find over-provisioned clusters.

Serverless has no clusters of yours to over-provision.

ON THE EXAM

Predictive optimization is a likely culprit here, and it bills under the serverless jobs SKU while running automatically.